

# P2P estructurado y no estructurado

Cómo millones de nodos sin servidor central  
logran encontrarse y compartir información.



DHT



IPFS



Gnutella



Freenet

# ¿Por qué P2P? · estructurado vs no estructurado

En **cliente-servidor**, un único servidor es cuello de botella, punto único de fallo y costoso de escalar. En **P2P**, cada nodo es a la vez cliente y servidor: la capacidad crece con la red y no hay un servidor que apagar.

## Estructurado

Topología lógica determinista (anillo, árbol XOR).  
Cada clave tiene un dueño calculable.

**O(log N)** garantizado · Chord, Kademlia

## No estructurado

Conexiones aleatorias. Búsqueda por flooding, gossip o heurística.

**O(N)** peor caso · Gnutella, Freenet

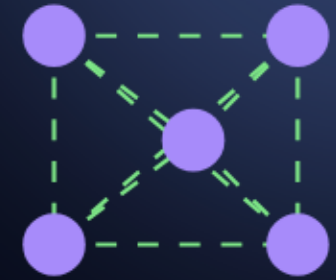
💡 BitTorrent e IPFS son híbridos: DHT estructurada para descubrir + intercambio descentralizado.

Cliente-servidor



1 punto de fallo

P2P



N rutas alternativas

# La idea de una DHT

¿Cómo encontrar quién guarda un fichero entre millones de nodos, sin un índice central?

- Claves y nodos viven en el **mismo espacio de IDs** (un hash SHA-1/256).
- La clave `k` la guarda el nodo cuyo ID está "**más cerca**" según una métrica.
- El enrutamiento descubre ese nodo en **log N saltos** conociendo solo a log N vecinos.

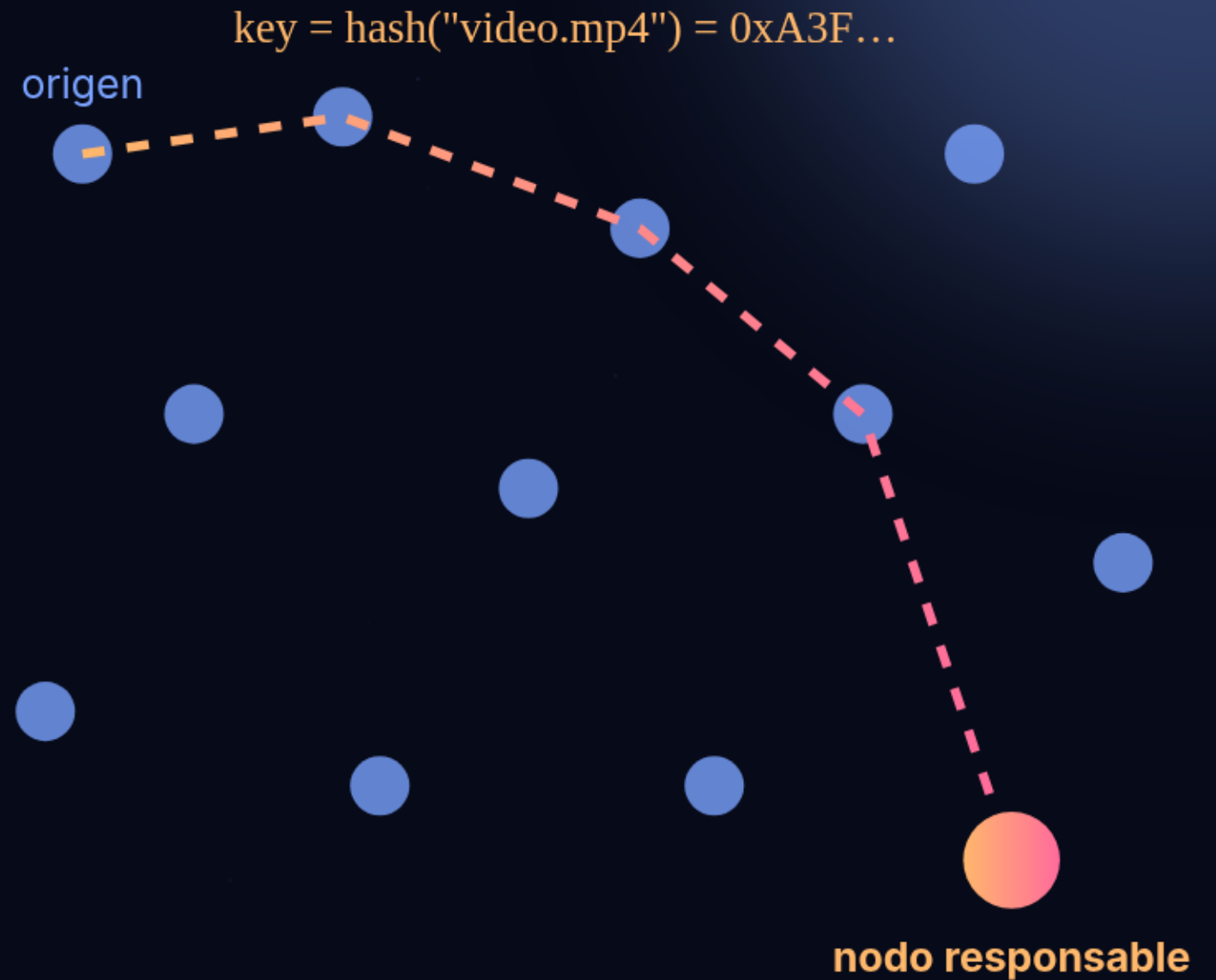
## API mínima

`put(key, value)` → almacena

`get(key)` → `value` → recupera

`find_node(id)` → ruta hasta el responsable

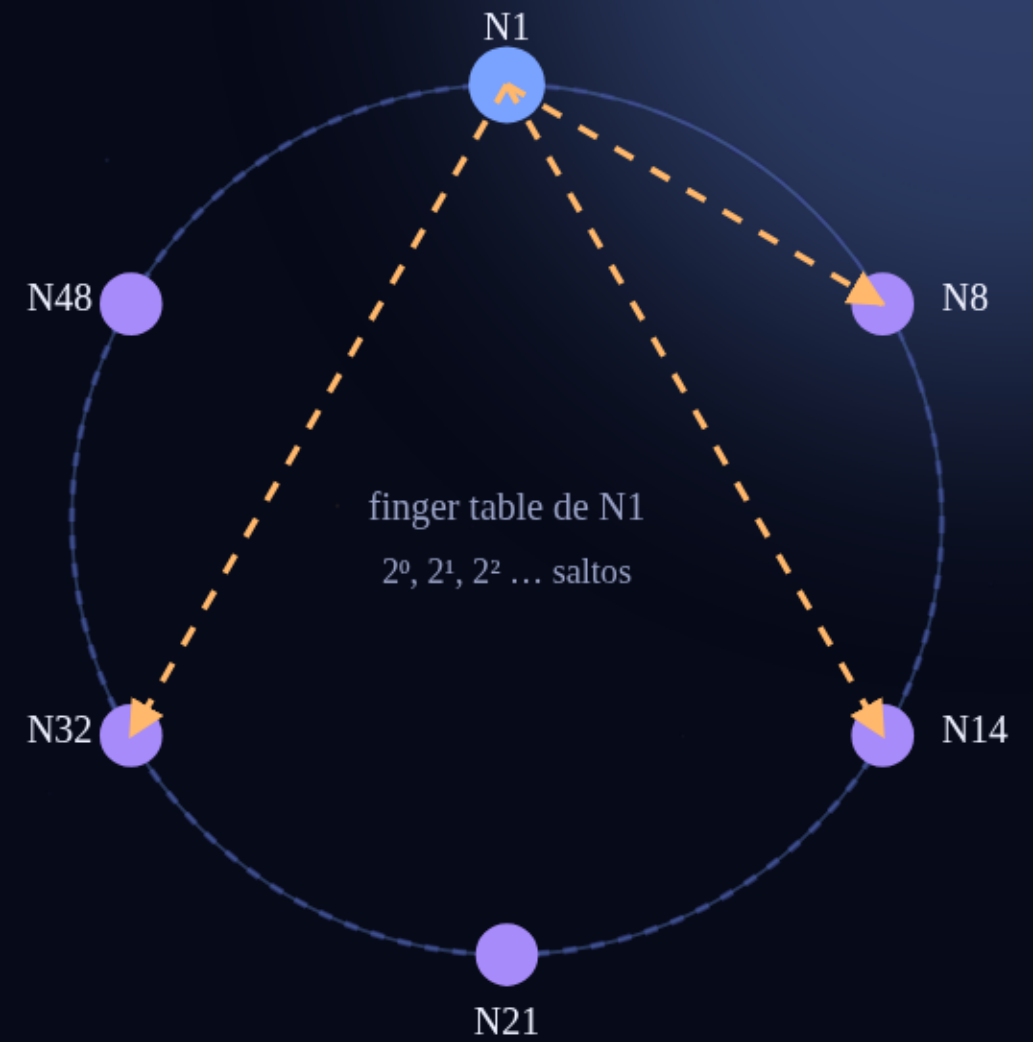
✂ Idea clave: cada salto **divide a la mitad** la distancia al objetivo. Es la misma intuición que una búsqueda binaria, distribuida por la red.



# Chord — DHT sobre un anillo

- IDs de  $m$  bits colocados en un **anillo módulo  $2^m$** .
- La clave  $k$  la guarda su **sucesor**: el primer nodo con  $ID \geq k$  al recorrer el anillo.
- Cada nodo mantiene una **finger table** con  $\log_2 N$  atajos a distancias  $2^0, 2^1, 2^2 \dots$  en el anillo.
- En cada salto se elige el atajo que **más se acerca** sin pasarse  $\rightarrow O(\log N)$ .
- `stabilize()` se ejecuta periódicamente para reparar el anillo cuando entran o salen nodos (*churn*).

**Analogía:** imagina buscar la página 500 de un libro. En lugar de leer una a una, tienes marcadores cada 1, 2, 4, 8, 16... páginas: saltas con el mayor que no se pase, y repites.



# Kademlia — XOR, k-buckets y lookup paralelo

- Distancia entre IDs =  $a \oplus b$ . Es **simétrica** (Chord no lo era), lo que simplifica el mantenimiento.
- Cada nodo guarda **160 k-buckets**: el bucket  $i$  contiene hasta  $k=20$  contactos cuyo ID comparte  $i$  bits de prefijo con el suyo.
- **LRU "invertida"**: se conservan los contactos *más antiguos*; estadísticamente sobreviven más → estabilidad sin `stabilize()`.
- **Lookup paralelo**: `FIND_NODE` a  $\alpha=3$  contactos a la vez; cada uno responde con los  $k$  más cercanos que conoce; se itera con los nuevos más cercanos hasta converger ( $\approx \log_2 N$  rondas).

Lo usan: BitTorrent (Mainline DHT), Ethereum (discv5), I2P, libp2p / IPFS.

**Tolerancia a fallos**: si uno de los  $\alpha=3$  no responde, los otros dos siguen. La red sobrevive a la rotación constante de nodos sin coordinador.



k-buckets agrupan contactos por prefijo común

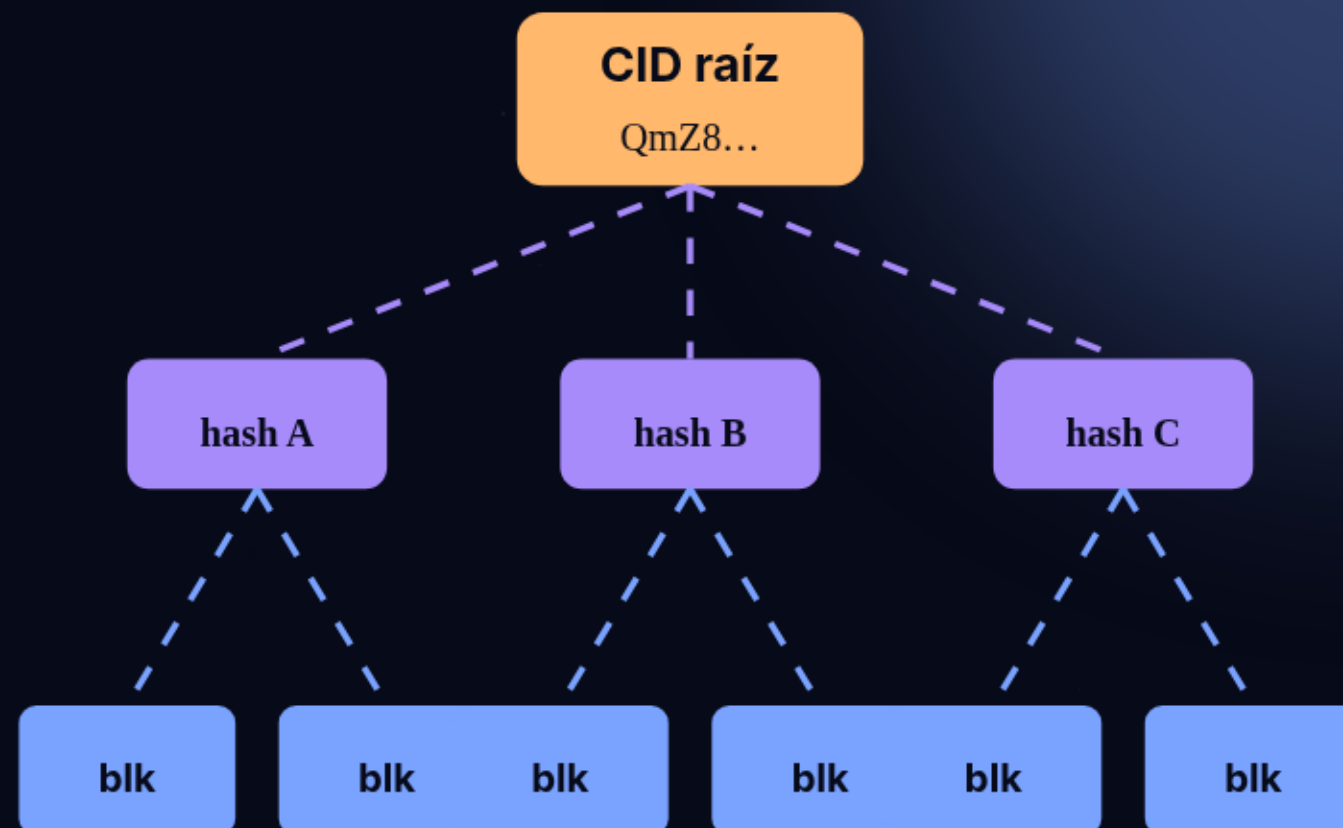
# IPFS — direccionamiento por contenido

HTTP pide "el archivo en *esta URL*".

IPFS pide "el archivo cuyo **hash** es este CID".

- El fichero se trocea en **bloques** (~256 KB) → cada bloque tiene su hash → forman un **Merkle DAG**.
- El **CID raíz** identifica el contenido de forma inmutable y *verificable*: cambia un bit, cambia el CID.
- **Kademlia (libp2p)** registra "yo tengo este CID" (*provider records*); **Bitswap** intercambia los bloques.
- **IPNS / DNSLink**: capa de nombres *mutables* sobre CIDs inmutables (firma o registro DNS).

**Trade-off:** el CID puro es verificable de extremo a extremo, pero inmutable. IPNS/DNSLink añaden mutabilidad a cambio de confiar en una firma o el DNS, y de más latencia.



Merkle DAG: cada nodo apunta al hash de sus hijos



# No estructurado: Gnutella y Freenet

## Gnutella — flooding + ultrapeers

- Un nodo envía **QUERY** a sus vecinos; estos lo reenvían **a todos** los suyos hasta agotar **TTL=7**.
- Cada mensaje lleva un **GUID** para detectar bucles.
- Coste: **O(N)** en el peor caso → no escala bien.
- v0.6 introduce **ultrapeers**: nodos potentes con mini-índice que aíslan a las hojas; usan **QRP** (filtros de Bloom) para no reenviar lo que saben que no está.

## Freenet — anonimato por diseño

- Enrutamiento **heurístico por similitud de claves**: cada nodo reenvía al vecino cuya clave se parece más a la solicitada.
- Cada salto **cachea** el resultado al pasarlo de vuelta → los contenidos populares se replican solos.
- El originador es **indistinguible** de un nodo intermedio → **negabilidad plausible**.
- Contenido **cifrado**: los nodos no saben qué almacenan. Modos *Opennet* / *Darknet*.

Sin índice global → robustez y simplicidad, pero búsqueda costosa. El precio de no imponer estructura.

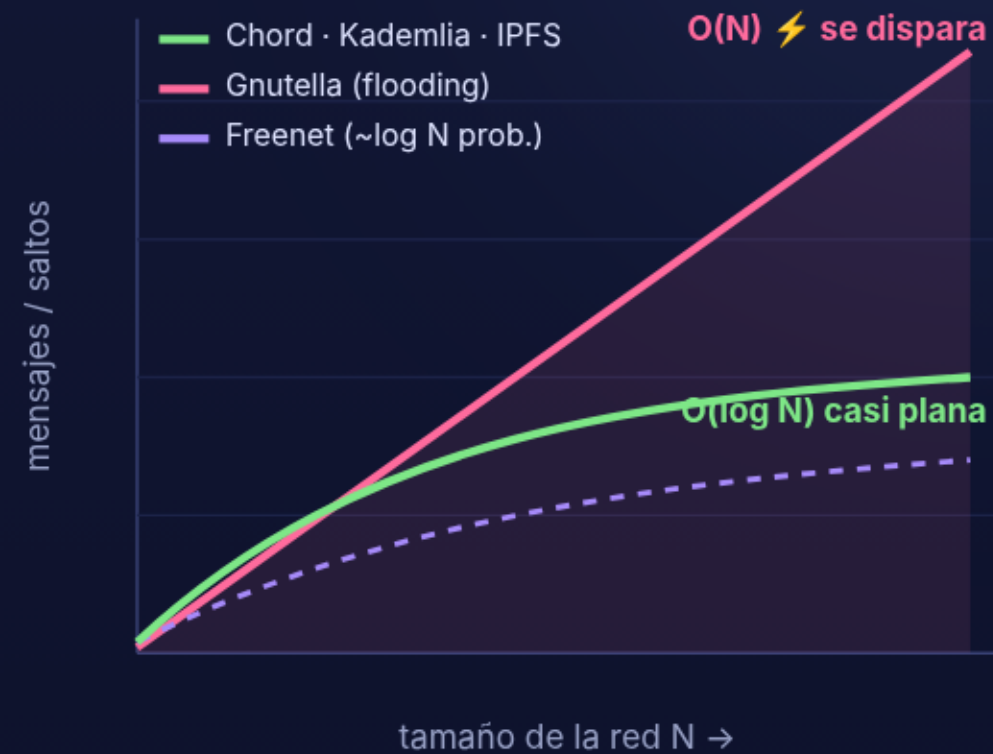
# Comparativa global

| Sistema  | Tipo               | Búsqueda            | Coste              | Fuerte en...           |
|----------|--------------------|---------------------|--------------------|------------------------|
| Chord    | Estructurado       | Finger table        | $O(\log N)$        | Simplicidad teórica    |
| Kademlia | Estructurado       | k-buckets paralelos | $O(\log N)$        | Resistencia al churn   |
| IPFS     | Híbrido (Kademlia) | Provider records    | $O(\log N)$        | Integridad y dedup.    |
| Gnutella | No estructurado    | Flooding + TTL      | $O(N)$             | Simplicidad y robustez |
| Freenet  | No estructurado    | Routing por clave   | $\sim O(\log N)^*$ | Anonimato y censura    |

Tres ejes de diseño: **topología** (impuesta vs emergente) · **coste de búsqueda** · **propiedad fuerte** que cada sistema prioriza. \*probabilístico.

## Cómo escala la búsqueda

Mensajes necesarios al crecer la red



Doblar la red apenas suma **1 salto** en estructurado, pero **duplica** el coste en flooding.



## 2 ideas que nos sorprendieron

### ① La regla LRU "al revés" de Kademlia

En la mayoría de cachés, lo *nuevo* desplaza a lo viejo. Kademlia hace lo contrario: conserva los contactos **más antiguos**.

Bucket lleno · llega un contacto nuevo



ping al más antiguo → si responde, el nuevo se rechaza

**Consecuencia:** la red se mantiene estable bajo churn extremo *sin* mantenimiento explícito. El propio tráfico refresca los buckets.

### ② IPFS no garantiza persistencia

Tener un CID *no* significa que el archivo siga existiendo. Si nadie lo "pinea", los nodos lo borran de su caché y el contenido **desaparece de la red** aunque la dirección siga siendo válida.

CID válido · pero contenido sin pin



con el tiempo, los nodos vacían su caché → **404**

**Consecuencia:** "direccionable" ≠ "almacenado". IPFS resuelve *integridad*, no *disponibilidad*: eso lo añaden Filecoin, pinning services o réplicas voluntarias.

# Cierre y fuentes

## Papers fundacionales

- Stoica et al. — *Chord*, SIGCOMM 2001.
- Maymounkov & Mazières — *Kademlia*, IPTPS 2002.
- Benet — *IPFS*, 2014.
- Ripeanu — *Gnutella Network Case Study*, P2P 2001.
- Clarke et al. — *Freenet*, 2001.

## Documentación

- [docs.ipfs.tech](https://docs.ipfs.tech) · documentación oficial IPFS.
- [github.com/libp2p/specs](https://github.com/libp2p/specs) · spec Kademlia.
- [www.bittorrent.org/beps](https://www.bittorrent.org/beps) · BEP-5 (Mainline DHT).
- [freenetproject.org](https://freenetproject.org) · proyecto Freenet/Hyphanet.
- Tanenbaum & Van Steen — *Distributed Systems*, 3ª ed., cap. 5.

### PREGUNTA 1 · KADEMLIA

Cuando un k-bucket está lleno y llega un contacto nuevo, ¿a quién conserva la red y por qué eso la hace resistente al *churn*?

### PREGUNTA 2 · IPFS

Tener el CID de un archivo, ¿garantiza poder descargarlo siempre? ¿Qué asegura el CID, qué no, y cómo se consigue lo que falta?

¡Gracias! · ¿Preguntas?